Find That Book

A Technical Take-Home Challenge

Overview

Build a small application for a library discovery workflow. Given a messy plain-text query containing a title, author, keywords, or some combination of these, identify the book or books it most likely refers to and present the results in a simple modern UI backed by a .NET Web API.

This project is designed to show how you approach a realistic product problem: working with imperfect input, integrating a third-party API, deciding where AI is useful, handling ambiguous data, and making pragmatic engineering tradeoffs.

> **Timebox matters**
>
> Please spend approximately 4-6 hours. We care more about your decisions, prioritization, and ability to explain tradeoffs than about completing every possible enhancement.

Core Expectations

The following are the expected scope of the submission. A strong solution should cover these areas without needing to be production-complete.

1. Accept messy plain-text queries

- Support sparse input such as only a title or only an author (for example: "dickens" or "tale two cities").
- Support noisy input containing extra words or mixed title/author information.
- Handle ambiguity well enough to return useful candidates rather than assuming every query has one obvious answer.

2. Use AI for structured interpretation

- Use an LLM to extract or normalize useful information from the query, such as title, author, and keywords.
- Keep the AI integration purposeful. We are interested in what you delegate to the model, what you keep deterministic, and why.
- Return a short, grounded explanation for each candidate describing why it matched.

3. Search Open Library for candidates

- Use Open Library APIs to search for candidate books and fetch additional work or author details when useful.
- Normalize enough input and result data to handle common differences in case, punctuation, subtitles, and partial names.
- Account for Open Library data quality issues rather than assuming every author_name value is a primary author.

4. Rank and return useful results

- Apply a sensible matching strategy that considers title and author evidence.
- Prefer canonical or primary-author matches over weaker contributor-only matches when the data supports that distinction.

- Return an ordered list of the best candidates. If there is no clear winner, returning several plausible results is appropriate.

5. Provide a simple end-to-end product

- Expose the functionality through a .NET Web API using the latest stable .NET SDK available when you start the project.
- Provide a small UI that allows a user to enter a query and understand the returned candidates.
- Handle common failure states gracefully, including invalid input, external API failures, and no useful matches.

6. Demonstrate engineering quality

- Include focused tests for the behavior you consider most important or risky.
- Keep the codebase understandable and reasonably organized for the size of the exercise.
- Document important assumptions and tradeoffs in the README.

Optional Depth

> **These are not required**
>
> Use these only if they are the best way to demonstrate your strengths or improve the product within the timebox. We do not expect every candidate to implement these items.

- LLM-based re-ranking after deterministic candidate retrieval.
- More sophisticated normalization for punctuation, diacritics, subtitles, aliases, or misspellings.
- De-duplication across editions or works, or deeper canonical-work resolution.
- Advanced contributor-vs-primary-author resolution using additional Open Library endpoints.
- Caching, retries, resilience policies, observability, or other production-oriented API concerns.
- A richer UI, accessibility refinements, loading states, or thoughtful interaction design beyond the basic workflow.
- Additional automated test coverage, contract tests, or test doubles around external dependencies.
- A deployed demo or hosted version of the project.

A note on scope: Choosing not to implement an optional item can be a good decision. If you deliberately leave something out, briefly explain what you would do next and why.

Data Quality Note

Open Library may list contributors such as illustrators, editors, or adaptors alongside primary authors. Your matcher should avoid treating every listed name as equally strong evidence. Where practical, use canonical work data to identify primary authors, and make the explanation reflect the evidence you actually used.

Return Results

For each returned candidate, include enough information for a user to understand the result:

- Book title
- Primary author or authors, where available
- First publish year, where available
- Relevant Open Library identifiers or links
- Cover image, if readily available

- A concise explanation of why the result matched the query

Numeric confidence scores are not required. The ordering and explanation should communicate the strength of the match.

```
{
  "title": "The Hobbit",
  "author": "J.R.R. Tolkien",
  "first_publish_year": 1937,
  "explanation": "Strong title and primary-author match."
}
```

Submission Guidelines

- Share a GitHub repository link containing your code.
- Include clear setup and run instructions. The project should use the latest stable .NET SDK available when you begin the exercise.
- Provide instructions for configuring the AI provider and required API keys. Gemini is a reasonable default, but another model/provider is fine if documented.
- Include a README covering your implementation approach, key assumptions or design decisions, testing strategy, features completed, and what you would improve with more time.
- A live deployment is welcome but optional.

Time Expectation

Please spend approximately 4-6 hours on this challenge. Stop when the timebox is reached, even if there are additional improvements you could make. We expect tradeoffs and unfinished ideas; use the README to tell us what you prioritized and what you would do next.

Be prepared to walk us through the project and discuss your decisions. We are interested in how you approached the problem at least as much as the final feature set.